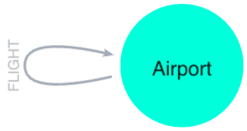
1-Projet de Romain Groult & Alban Talagrand

Ils ont pris une base de donnée sur un Dataset de vols américain :

Dataset Kaggle “2015 Flight Delays and Cancella- tions” (US Department of Transportation) :

Car pour eux la dataset IDFM était trop complexe





Modèle de graphe Neo4j Nœuds :

— Airport (313) : Code IATA, nom, ville, état, coordonnées GPS — Airline (14) : Code IATA, nom de la compagnie

Relations :

— FLIGHT (107 230) : Propriétés :
— departure_ts / arrival_ts : Timestamps
— distance : Distance en miles
— delay : Retard au départ en minutes (peut être négatif) — airline : Code de la compagnie

Contraintes :
— Unicité des codes IATA (airports et airlines)

— Index sur departure_ts, distance, delay pour optimiser les requêtes

Choix de modélisation : Chaque vol est une relation unique, identifiée par son timestamp. Cela évite le problème des multi-edges : ANC->SEA à 23h54 et ANC->SEA à 08h30 sont deux relations distinctes, ce qui simplifie les requêtes de chemins temporels.

Modèle relationnel PostgreSQL

airlines
- iata_code (PK)
- name

airports
  - iata_code (PK)
  - name, city, state, country
  - latitude, longitude
flights
  - id (PK)
  - source (FK -> airports.iata_code)
  - target (FK -> airports.iata_code)
  - airline (FK -> airlines.iata_code)
  - departure_ts, arrival_ts
  - distance, delay
  - CHECK (source != target AND distance > 0)

Index sur source, target, departure_ts pour optimiser les requêtes récursives.

Comparaison Cypher 5 (NOT EXISTS) et Cypher 25 (allReduce) :

Cypher 5 :

Stratégie : Génère TOUS les chemins (319 631), puis filtre a posteriori.

Résultats :

— Temps : ~1,5 seconde
— DBHits:16340381
— Rows traitées : 320 799

Cypher 25 :

Stratégie : Filtre PENDANT la traversée grâce à l'opérateur Trail. Résultats :

— Temps : ~3,5 ms
— DBHits:39904
— Rows traitées : 9 290

Gain :

— Temps : 1 496,5ms de moins
— DB Hits : 16300477 de moins
— Rows traitées : 311 509 de moins

Premiere comparaison on constate que Cypher 25 est plus efficace que Cypher 5.

Problème : Trouver l'itinéraire JFK->DAY minimisant la distance totale. Le plus court chemins pondérés

Cypher 5 (shortestPaths) pas pondéré :

— Route : [JFK, ATL, DAY]
— Distance : 1192 miles
— Temps : 15 ms

Cypher 25 (pondéré) :
— Route : [JFK, BWI, DAY]
— Distance : 590 miles (optimal)
— Temps : 293 ms

Explosion combinatoire :

— *1..2 (2 sauts max) : 293 ms
— *1..3 (3 sauts max) : 137 secondes — *1..4 (4 sauts max) : timeout

Au-delà de 2-3 sauts, la requête devient inutilisable.

GDS Dijkstra :
— Route : [JFK, BWI, DAY]
— Distance : 590 miles (optimal garanti)
— Temps : 37 ms

Comparaison :
— Cypher 5 : Non pondéré, pas optimal
— Cypher 25 : Timeout au-delà de 2-3 sauts
— GDS Dijkstra : Toujours performant, optimal, scalable

Conclusion :

Cypher 25 vs Cypher 5 : Cypher 25 confirme sa supériorité pour éviter l'explosion combinatoire grâce au filtrage durant la traversée (allReduce) et simplifie le code via les quantified patterns.

GDS reste insipensable car Cypher pur manque de structure optimisée (priority queue) au dela de 2-3 saut time out

Donc

SQL : Bien que capable, le SQL récursif est jugé peu lisible et moins adapté pour l'analyse de chemins complexes.

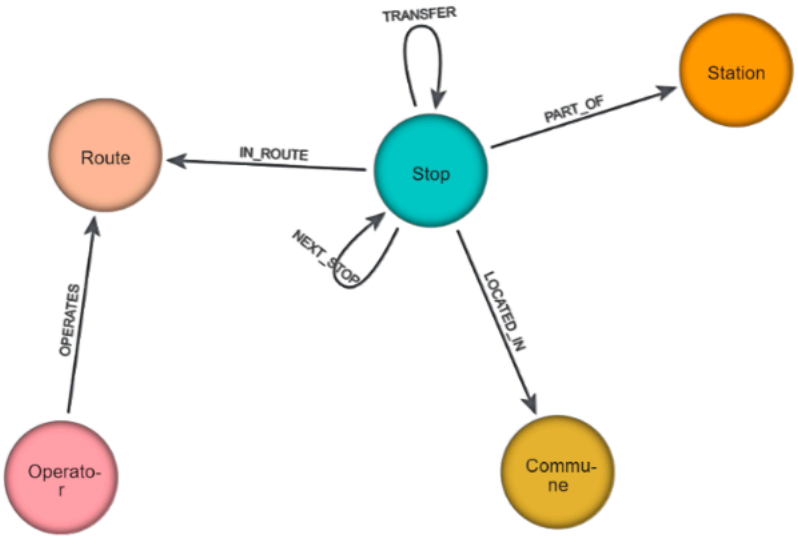
2- Projet de Paris Albin et Yazici Servan

Utilisation de données open source sur le site d'IDFM-Mobilité.

API des arrets et lignes associés : Cette API fournit des informations détaillées

sur les arrêts de transport en commun et les lignes associées en Île-de-France.

API des Tracés des lignes de transport en commun d'Île-de-France : Cette API offre des données sur les tracés des différentes lignes de transport en commundans la région Île-de-France.



• Nœuds :
o (:Stop) : Représente un point d'arrêt physique du réseau.
▪ Propriétés : id, commune, insee, lat, lon, name, stopId
o (:Station) : Représente un ensemble de stops
▪ Propriétés : id, commune, insee, lat, lon, name, stationId
▪ (lat,lon) = moyenne de tous les stops appartenant à la station
o (:Route) : Représente une ligne de transport
▪ Propriétés : id, longName, mode, operator, routeId, shortName
o (:Operator) : Représente l'opérateur d'une ligne de transport
▪ Propriétés : id, name
o (:Commune) : Représente la commune d'un stop
▪ Propriétés : id, insee, name

Comparaison Cypher 5 , Cypher 25, SQL :

Cas simples :

- Cypher 5 et 25 ont une performance similaire pour un calcul de plus court chemin standard (shortedPath) mais le cout de Cypher 25 a un cout legement supplémentaire pour respecter le standard GQL.

- SQL ne parvient pas a certains moment a terminer

Filtrage complexe et contraintes :

- Cypher 5 (Post-Filtering) : strategie genere puis filtre. Cela explose le graphe a l'aveugle et genere tous les chemins possible avant de filtrer a la fin (where) -> consommation massive de ressources (3,5 millions de DBHitbs pour une requete)

-Cypher 25 (Early Pruning) : Stratégie élagage précoce grace aux Quantified Path Pattern, le filtre est intégré directement dans la bouclde de travaersée les branches invalides sont abandonnées immédatiement -> Cypher 25 génère 4,6 fois moins d'acces disque que Cypher 5 sur des requetes complexes

- SQL : Dégradation critique et ne termine souvent pas (timeout) faute de protection efficace contre l'explosion combinatoire des choix possible

Détection de cycles (Unicité des nœuds) :

Cypher 5 : Doit matérialiser les chemins avec boucles vant de pouvoir vérifier l'unicité, ce qui est tres couteux

Cypher 25 : Utilise ALLREDUCE pour vérifier l'unicité a chaque étape de la traversée, surpassant Cypher 5 en évitant l'exploration de branche inutiles

SQL : Performance intéressante grace a un effet coupe-circuit dans la requete recursive, qui élimine les cycles avant matérialisation, parfois plus réponds plus rapique que Cypher 5

Limites communes et nécessité de GDS :

Si on veut minimiser le temps de trajet SQL, Cypher echoyent car ils tentent d'énumerer tous les chemins pour trier ensuite, souvent finissent en timeout sur un graphe denses, la solution GDS comme l'algo Dikjstra est impérative pour obtenir des résultats performants.

Conclusion :

Cypher 25 > Cypher 5 grace a sa capacité de filtrage pendant la traversér (Early Pruning) et ses nouvelles fonctionnalités (ALLREDUCE, QGP)
SQL moins adapté pour les syntaxes lourdes ; gestion complexe de la récursiviété et performances instables sur les chemins profonds.

3- Projet de Chris ESSOMBA et Ivan BANFOU

Données et Modélisation

Dataset : Capital Bikeshare (trajets de vélos à New York).
Volume : Environ 429 stations (nœuds) et 5 000 trajets (relations) pour les tests limités.

Modélisation Neo4j :

- Nœuds : Station (id, nom, lat, long).
- Relations : TRIP (propriétés
: duration_minutes, started_at, rideable_type...).

Modélisation PostgreSQL :
Tables station et trip avec indexation appropriée.

Comparaison Cypher 5 , Cypher 25, SQL :

Cypher 5 (WHERE NOT EXISTS) :

Stratégie : Double pattern matching pour exclure les cas non valides.
Résultat : Très performant (70 ms), car permet l'élagage précoce sans matérialiser les listes.

Cypher 5 (Reduce) :

Stratégie : Matérialise toute la liste des relations avant de vérifier la condition.

Résultat : Échec / Timeout. Explosion computationnelle due à la matérialisation complète.

Cypher 25 (allReduce) :

Stratégie : Évaluation incrémentale. Arrête le parcours dès que la condition est fausse (Pruning).

Résultat : Fonctionne (1 min 55 seconde), mais lent car l'expansion génère énormément de chemins candidats (188M d'accès base) avant le filtrage.

SQL (Recursive CTE) :

Résultat : Plus rapide que Cypher 25 (environ 10,5 seconde) pour ce cas précis, mais provoque une écriture sur disque (temp files) indiquant une saturation mémoire.

Requête : Simulation d'Automate Fini (Alternance de types) :

Cypher 5 (Reduce) :

- Temps : ~1,86s.
- Défaut : 4 millions d'accès base. Obligé de tout explorer avant de filtrer

Cypher 25 (allReduce + QPP) :

- Temps : ~0,94s.

- Avantage : 1,36 million d'accès base. Cypher 25 est 2x plus rapide et 3x plus économe en accès grâce au filtrage pendant la traversée.

SQL :

- Temps : ~0,2s.
- Note : Performance biaisée par le LIMIT. Le plan d'exécution montre que PostgreSQL ne calcule que la branche de longueur 1 et "court-circuite" les jointures complexes (longueurs 2 et 3 non exécutées).

Requête : Plus court chemin avec somme cible (Subset Sum)

- **Problème :** Plus court chemin dont la somme des durées vaut exactement 30 minutes.
- **Cypher 5 (allShortestPaths) :**
  - Temps : 147 secondes.
  - Défaut : Calcule d'abord tous les chemins non pondérés (BFS), puis filtre la somme à la fin. Inefficace.
- **Cypher 25 (allShortest + allReduce) :**
  - Résultat : Timeout. L'ajout de allReduce ne compense pas l'explosion combinatoire de l'exploration non guidée par les poids.
- **GDS (Dijkstra) :**
  - Temps : 2,6 secondes.
  - Avantage : Intègre nativement le poids (duration_minutes) dans l'exploration. Nettement supérieur (x50 plus rapide).

Conclusion :

Comme les autres projets on remarque que Cypher 5 < Cypher 25 pour les logiques complexes grace a l'ajout de allreduce
SQL instable
Nécessité GDS pour les graphes pondérés ou chemins denses

4- Projet de Mohamed Laidouni et Eliot Friedmann

Cypher 25 > Cypher 25 et change la donne pour les requêtes dépendantes de l'état du chemin (temps, alternance) grâce à l’élagage précoce, passant de minutes à millisecondes.

SQL reste très compétitif (parfois égal à Cypher 25) tant que le modèle relationnel est bien indexé (Index Only Scans), mais souffre structurellement sur les jointures multiples.

Le GDS pour les chemins pondérés. Cypher pur (même 25) ne peut pas imiter efficacement Dijkstra.

5- Projet de Lin Michel et Ba Alpha

Cypher 25 : Apporte une expressivité structurelle majeure (QPP) qui remplace avantageusement le filtrage de listes. Cependant, l'optimisation de allReduce semble encore immature (pas de pruning agressif observé).

SQL : Reste supérieur pour l'analytique globale (fenêtrage), mais devient un enfer syntaxique et performatif pour la navigation complexe (gestion manuelle des cycles).

GDS : Confirmé comme indispensable pour les problèmes d'optimisation (chemin pondéré). Le fossé entre le requêtage transactionnel (Cypher) et l'algorithmique (GDS) est immense (x400 en vitesse sur le pondéré).

6- Projet de Bendali Manal et Barkan Ines

Cypher 25 : Apporte une stabilité cruciale sur les chemins longs (benchmark 300 sauts) et simplifie la syntaxe des filtres complexes (QGP).

SQL : Reste le maître des agrégations statistiques globales (AVG, COUNT) mais perd dès qu'il s'agit de navigation récursive (chemins).

Frontière GDS : Le projet confirme la frontière nette entre le requêtage (Cypher) et l'algorithmique (GDS). Pour l'optimisation combinatoire (Dijkstra), passer par une projection mémoire GDS est la seule solution viable.

7- Projet de You Badar et De Badarou

Cypher 25 (QGP) : Révolutionne l'écriture de requêtes complexes (motifs répétés/alternés) qui étaient impossibles en Cypher 5.

Limites de allReduce : Sur la monotonie simple, il ne bat pas forcément un NOT EXISTS bien placé car il n'interrompt pas toujours l'itération prématurément.

GDS vs Cypher : Le rapport insiste sur la complémentarité. Cypher est pour l'interrogation (Pattern Matching), GDS est pour l'analyse (Algorithmes itératifs). Tenter de faire du PageRank ou du Dijkstra en Cypher pur est une "erreur d'architecture".

SQL : Reste très pertinent pour la navigation récursive simple grâce à l'optimisation des jointures, mais devient verbeux et moins expressif pour les motifs complexes.

8- Projet de You Badar et De Badarou

Cypher 25 : Domine pour l'exploration complexe (QPP, Monotonie) grâce à sa syntaxe concise et son filtrage précoce (allReduce).

C**ypher 5 :** Reste paradoxalement meilleur pour le BFS simple (Plus court chemin non pondéré) grâce à son implémentation bidirectionnelle mature.

SQL : Très solide pour les agrégations et certaines astuces (Random Walk via Lateral Join), mais souffre de verbosité et de lourdeur mémoire sur les récursions profondes.

GDS : Confirmé comme l'outil obligatoire dès qu'on sort de la navigation transactionnelle pour faire de l'algorithmique (Dijkstra, Centralité).